硅谷大脑 编译翻译 · Silicon Valley Brain

LLM Wiki

A pattern for building personal knowledge bases using LLMs
使用大语言模型构建个人知识库的方法论

原文：Andrej Karpathy | gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

> This is an idea file, designed to be copy-pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / Pi). Its goal is to communicate the high-level idea; your agent will build out the specifics in collaboration with you.

这是一份思路文件，设计上可以直接复制粘贴到你自己的 LLM Agent（例如 OpenAI Codex、Claude Code、OpenCode / Pi 等）。它的目标是传达高层次的核心思路，具体细节由你和 Agent 共同协作落地。

The core idea / 核心思路

> Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

大多数人使用 LLM 处理文档的方式都是 RAG：上传一批文件，提问时让模型检索相关片段并生成答案。这套方案能用，但模型每次回答都是从零开始重新发现知识，没有任何积累。如果你的问题需要综合五篇文档的内容，模型就得每次重新找、重新拼。什么都没有建立起来。NotebookLM、ChatGPT 的文件上传功能，以及绝大多数 RAG 系统，都是这个逻辑。

> The idea here is different. Instead of just retrieving from raw documents at query time, the LLM **incrementally builds and maintains a persistent wiki** — a structured, interlinked collection of markdown files that sits between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then *kept current*, not re-derived on every query.

这里提出的思路截然不同。不是在提问时才去检索原始文档，而是让 LLM 持续地**增量构建并维护一个永久性的 Wiki**——一套结构化、相互链接的 Markdown 文件，横亘在你和原始资料之间。每当加入一份新资料，LLM 不只是把它编入索引留待日后检索，而是真正读懂它、提取关键信息，并将其融入已有的 Wiki——更新实体页面、修订主题摘要、标注新旧内容的矛盾之处，不断强化或修正整体认知。知识只需编译一次，之后持续保持更新，而不是每次提问都重新推导一遍。

> This is the key difference: **the wiki is a persistent, compounding artifact.** The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

这正是关键所在：**Wiki 是一个持久的、复利式的知识资产。**交叉引用已经在那里了。矛盾已经被标记出来了。综合性的结论已经反映了你读过的所有内容。每加入一份新资料、每提一个新问题，Wiki 就变得更丰富一分。

> You never (or rarely) write the wiki yourself — the LLM writes and maintains all of it. You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time. In practice, I have the LLM agent open on one side and Obsidian open on the other. The LLM makes edits based on our conversation, and I browse the results in real time — following links, checking the graph view, reading the updated pages. Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase.

小红书
小红书号：95578602331

你几乎不需要亲自动手写 Wiki——一切都由 LLM 来写和维护。你负责筛选资料、探索方向、提出好问题；LLM 负责所有繁琐的工作——摘要、交叉引用、归档，以及让知识库真正好用所需的各种维护工作。在实际使用中，一边开着 LLM 对话窗口，一边打开 Obsidian。LLM 在对话中做出修改，实时在 Obsidian 里浏览结果——顺着链接跳转、查看知识图谱、阅读更新后的页面。Obsidian 是 IDE，LLM 是程序员，Wiki 是代码库。

> This can apply to a lot of different contexts. A few examples:

这套模式可以应用于很多场景，举几个例子：

> - **Personal:** tracking your own goals, health, psychology, self-improvement — filing journal entries, articles, podcast notes, and building up a structured picture of yourself over time.

- **个人成长：**追踪自己的目标、健康状况、心理状态、自我提升——把日记、文章、播客笔记归档进来，逐步建立起一幅关于自己的结构化图景。

> - **Research:** going deep on a topic over weeks or months — reading papers, articles, reports, and incrementally building a comprehensive wiki with an evolving thesis.

- **深度研究：**围绕某个课题持续钻研数周乃至数月——阅读论文、文章、报告，增量式地构建一个有完整论点演进的综合性 Wiki。

> - **Reading a book:** filing each chapter as you go, building out pages for characters, themes, plot threads, and how they connect. By the end you have a rich companion wiki. Think of fan wikis like Tolkien Gateway — thousands of interlinked pages built by a community of volunteers over years.

- **读书笔记：**每读完一章就归档一次，为人物、主题、情节线索建立页面，并梳理它们之间的联系。读完全书，你就拥有了一个丰富的配套 Wiki。想想托尔金百科（Tolkien Gateway）这样的粉丝 Wiki——由社区志愿者历时多年构建，成千上万个相互链接的页面。

> - **Business/team:** an internal wiki maintained by LLMs, fed by Slack threads, meeting transcripts, project documents, customer calls. Possibly with humans in the loop reviewing updates.

- **企业/团队：**一个由 LLM 维护的内部 Wiki，输入来源包括 Slack 消息、会议记录、项目文档、客户通话。可以加入人工审核环节。

> - **Competitive analysis, due diligence, trip planning, course notes, hobby deep-dives** — anything where you're accumulating knowledge over time and want it organized rather than scattered.

- **竞争对手分析、尽职调查、旅行规划、课堂笔记、兴趣爱好的深度探索**——任何需要持续积累知识、并希望它有条理而不是一盘散沙的场景，都适用。

Architecture / 系统架构

> There are three layers:

整个系统分为三层：

> - **Raw sources** — your curated collection of source documents. Articles, papers, images, data files. These are immutable — the LLM reads from them but never modifies them. This is your source of truth.

- **第一层：原始资料** — 你精心收集的原始文档，包括文章、论文、图片、数据文件。这一层是只读的——LLM 只读取，不修改。这是你的事实来源。

> - **The wiki** — a directory of LLM-generated markdown files. Summaries, entity pages, concept pages, comparisons, an overview, a synthesis. The LLM owns this layer entirely. It creates pages, updates them when new sources arrive, maintains cross-references, and keeps everything consistent. You read it; the LLM writes it.

- **第二层：Wiki** — 一个由 LLM 生成的 Markdown 文件目录，包含摘要、实体页面、概念页面、对比分析、概览和综合性结论。这一层完全由 LLM 负责：它创建页面、在新资料加入时更新页面、维护交叉引用、保持内容的一致性。你负责阅读，LLM 负责写作。

> - **The schema** — a document (e.g. `CLAUDE.md` for Claude Code or `AGENTS.md` for Codex) that tells the LLM how the wiki is structured, what the conventions are, and what workflows to follow when ingesting sources, answering questions, or maintaining the wiki. This is the key configuration file — it's what makes the LLM a disciplined wiki maintainer rather than a generic chatbot. You and the LLM co-evolve this over time as you figure out what works for your domain.

- **第三层：Schema（规范文档）** — 一份配置文件（例如 Claude Code 用的 `CLAUDE.md`，或 Codex 用的

小红书
小红书号：95578602331

`AGENTS.md`），告诉 LLM Wiki 的结构是什么、约定规范是什么，以及在录入资料、回答问题或维护 Wiki 时应遵循什么工作流程。这是最关键的配置文件——正是它让 LLM 成为一个有纪律的 Wiki 维护者，而不只是一个通用聊天机器人。你和 LLM 会随着时间的推移共同完善它，在实践中摸索出最适合你所在领域的方式。

Operations / 三种核心操作

> **Ingest.** You drop a new source into the raw collection and tell the LLM to process it. An example flow: the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log. A single source might touch 10–15 wiki pages. Personally I prefer to ingest sources one at a time and stay involved — I read the summaries, check the updates, and guide the LLM on what to emphasize. But you could also batch-ingest many sources at once with less supervision. It's up to you to develop the workflow that fits your style and document it in the schema for future sessions.

录入（Ingest） 你把一份新资料放入原始资料库，告诉 LLM 来处理它。一个典型的工作流程是：LLM 阅读资料、与你讨论关键要点、在 Wiki 里写一篇摘要页面、更新索引、更新 Wiki 中相关的实体和概念页面，并在日志中添加一条记录。一份资料可能会涉及 10 到 15 个 Wiki 页面。个人倾向于一次录入一份资料，并全程参与其中——阅读摘要、检查更新、引导 LLM 重点关注哪些内容。当然也可以批量录入多份资料，减少干预。具体采用什么工作流程，取决于你自己的习惯，记得在 Schema 中记录下来，供后续使用。

> **Query.** You ask questions against the wiki. The LLM searches for relevant pages, reads them, and synthesizes an answer with citations. Answers can take different forms depending on the question — a markdown page, a comparison table, a slide deck (Marp), a chart (matplotlib), a canvas. The important insight: **good answers can be filed back into the wiki as new pages.** A comparison you asked for, an analysis, a connection you discovered — these are valuable and shouldn't disappear into chat history. This way your explorations compound in the knowledge base just like ingested sources do.

查询（Query） 你向 Wiki 提问。LLM 搜索相关页面、阅读内容、综合作答并附上引用。答案的形式可以多种多样，根据问题而定——Markdown 页面、对比表格、幻灯片（Marp 格式）、图表（matplotlib）、画布等。重要的洞见在于：**好的回答可以作为新页面写回 Wiki。**你提出的某个对比分析、某个发现的联系——这些都是有价值的，不应该消失在聊天记录里。这样，你的探索过程就像录入的资料一样，在知识库中不断积累。

> **Lint.** Periodically, ask the LLM to health-check the wiki. Look for: contradictions between pages, stale claims that newer sources have superseded, orphan pages with no inbound links, important concepts mentioned but lacking their own page, missing cross-references, data gaps that could be filled with a web search. The LLM is good at suggesting new questions to investigate and new sources to look for. This keeps the wiki healthy as it grows.

检查（Lint） 定期让 LLM 对 Wiki 做一次健康检查，排查：页面之间的矛盾、被新资料推翻的陈旧说法、没有任何入链的孤立页面、被提及但缺少独立页面的重要概念、缺失的交叉引用、可以通过网络搜索填补的信息空白。LLM 擅长建议值得深入研究的新问题和值得寻找的新资料，这有助于 Wiki 在不断扩张的同时保持健康。

Indexing and logging / 索引与日志

> Two special files help the LLM (and you) navigate the wiki as it grows. They serve different purposes:

两个特殊文件帮助 LLM（和你）在 Wiki 扩大后依然能高效导航，二者用途不同：

> - **index.md** is content-oriented. It's a catalog of everything in the wiki — each page listed with a link, a one-line summary, and optionally metadata like date or source count. Organized by category (entities, concepts, sources, etc.). The LLM updates it on every ingest. When answering a query, the LLM reads the index first to find relevant pages, then drills into them. This works surprisingly well at moderate scale (~100 sources, ~hundreds of pages) and avoids the need for embedding-based RAG infrastructure.

- **index.md** 是内容导向的。它是整个 Wiki 的目录——每个页面都附有链接、一句话摘要，以及可选的元数据（如日期、资料来源数量），按类别组织（实体、概念、来源等）。LLM 在每次录入时更新它。回答问题时，LLM 先读索引找到相关页面，再深入阅读。在中等规模（约 100 份资料、数百个页面）下，这套方式效果出奇地好，也无需搭建基于向量嵌入的 RAG 基础设施。

> - **log.md** is chronological. It's an append-only record of what happened and when — ingests, queries, lint passes. A useful tip: if each entry starts with a consistent prefix (e.g. `## [2026-04-02] ingest | Article Title`), the log becomes parseable with simple unix tools — `grep "^## \[" log.md | tail -5` gives you the last 5 entries. The log gives you a timeline of the wiki's evolution and helps the LLM understand what's

小红书
小红书号：95578602331

been done recently.

- **log.md** 是时间导向的。它是一份只追加不修改的操作记录，记录发生了什么、发生在什么时候——包括录入、查询、检查等操作。一个实用技巧：如果每条记录以固定前缀开头（例如 `## [2026-04-02] ingest | 文章标题`），日志就变得可以用简单的 Unix 工具来处理——`grep "^## \[" log.md | tail -5` 就能列出最近 5 条记录。日志给你提供了 Wiki 演进的时间线，也帮助 LLM 了解最近做过什么。

Optional: CLI tools / 可选：命令行工具

At some point you may want to build small tools that help the LLM operate on the wiki more efficiently. A search engine over the wiki pages is the most obvious one — at small scale the index file is enough, but as the wiki grows you want proper search. **qmd** is a good option: it's a local search engine for markdown files with hybrid BM25/vector search and LLM re-ranking, all on-device. It has both a CLI (so the LLM can shell out to it) and an MCP server (so the LLM can use it as a native tool). You could also build something simpler yourself — the LLM can help you vibe-code a naive search script as the need arises.

随着使用深入，你可能希望构建一些小工具，帮助 LLM 更高效地操作 Wiki。最显而易见的是 Wiki 的搜索引擎——规模较小时，索引文件已经够用；但随着 Wiki 不断扩大，你会希望有一个真正的搜索功能。**qmd** 是一个不错的选择：它是一个本地 Markdown 文件搜索引擎，支持 BM25/向量混合搜索和 LLM 重排序，完全在本地设备上运行。它既有 CLI 接口（LLM 可以直接调用），也有 MCP 服务（LLM 可以将其作为原生工具使用）。你也可以自己构建更简单的工具——在需要的时候，LLM 可以帮你快速写一个简单的搜索脚本。

Tips and tricks / 实用技巧

- **Obsidian Web Clipper** is a browser extension that converts web articles to markdown. Very useful for quickly getting sources into your raw collection.

- **Obsidian Web Clipper** 是一个浏览器插件，可以将网页文章转换为 Markdown，对于快速将内容放入原始资料库非常有用。

- **Download images locally.** In Obsidian Settings → Files and links, set "Attachment folder path" to a fixed directory (e.g. `raw/assets/`). Then in Settings → Hotkeys, search for "Download" to find "Download attachments for current file" and bind it to a hotkey (e.g. Ctrl+Shift+D). After clipping an article, hit the hotkey and all images get downloaded to local disk. Note that LLMs can't natively read markdown with inline images in one pass — the workaround is to have the LLM read the text first, then view some or all of the referenced images separately.

- **本地下载图片：**在 Obsidian 设置 → 文件与链接中，将"附件文件夹路径"设置为一个固定目录（如 `raw/assets/`）；然后在设置 → 快捷键中搜索"Download"，找到"下载当前文件的附件"，绑定一个快捷键（如 Ctrl+Shift+D）。剪藏文章后按快捷键，所有图片就会下载到本地。注意：LLM 无法一次性读取内嵌了图片的 Markdown 文件——变通方法是让 LLM 先读文本，再单独查看部分或全部图片以获取补充信息。

- **Obsidian's graph view** is the best way to see the shape of your wiki — what's connected to what, which pages are hubs, which are orphans.

- **Obsidian 的图谱视图**是查看 Wiki 全貌的最佳方式——哪些页面相互连接，哪些页面是枢纽，哪些是孤岛。

- **Marp** is a markdown-based slide deck format. Obsidian has a plugin for it. Useful for generating presentations directly from wiki content.

- **Marp** 是一种基于 Markdown 的幻灯片格式，Obsidian 有对应插件，可以直接从 Wiki 内容生成演示文稿。

- **Dataview** is an Obsidian plugin that runs queries over page frontmatter. If your LLM adds YAML frontmatter to wiki pages (tags, dates, source counts), Dataview can generate dynamic tables and lists.

- **Dataview** 是一个 Obsidian 插件，可以对页面的 YAML 前置元数据运行查询。如果你的 LLM 为 Wiki 页面添加了前置元数据（标签、日期、来源数量等），Dataview 可以生成动态表格和列表。

- The wiki is just a **git repo** of markdown files. You get version history, branching, and collaboration for free.

- Wiki 本质上就是一个 **Git 仓库**，全是 Markdown 文件。版本历史、分支管理、多人协作，全部开箱即用。

Why this works / 为什么这套方法有效

The tedious part of maintaining a knowledge base is not the reading or the thinking — it's the bookkeeping. Updating cross-references, keeping summaries current, noting when new data contradicts old claims,

小红书
小红书号：95578602331

> maintaining consistency across dozens of pages. Humans abandon wikis because the maintenance burden grows faster than the value. LLMs don't get bored, don't forget to update a cross-reference, and can touch 15 files in one pass. The wiki stays maintained because the cost of maintenance is near zero.

维护知识库最繁琐的部分，不是阅读，也不是思考——而是**记账**。更新交叉引用、保持摘要的时效性、标注新数据与旧观点的矛盾、维护数十个页面之间的一致性。人们放弃维护 Wiki，正是因为维护成本的增速超过了它带来的价值。LLM 不会感到无聊，不会忘记更新某个交叉引用，可以在一次操作中同时修改 15 个文件。Wiki 得以持续维护，是因为维护的成本几乎为零。

> The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else.

人的工作是：筛选资料、指引分析方向、提出好问题、思考这一切意味着什么。LLM 的工作是：其他一切。

> The idea is related in spirit to Vannevar Bush's Memex (1945) — a personal, curated knowledge store with associative trails between documents. Bush's vision was closer to this than to what the web became: private, actively curated, with the connections between documents as valuable as the documents themselves. The part he couldn't solve was who does the maintenance. The LLM handles that.

这个想法在精神上与 Vannevar Bush 在 1945 年提出的"记忆延伸机器"（Memex）一脉相承——一个私人的、经过主动筛选的知识库，文档之间存在联想式的关联路径。Bush 的愿景比后来的万维网更接近这个思路：私密的、主动维护的，文档之间的联系与文档本身同等重要。他当年唯一没有解决的问题，是谁来负责维护。LLM 解决了这个问题。

Note / 附注

> This document is intentionally abstract. It describes the idea, not a specific implementation. The exact directory structure, the schema conventions, the page formats, the tooling — all of that will depend on your domain, your preferences, and your LLM of choice. Everything mentioned above is optional and modular — pick what's useful, ignore what isn't. For example: your sources might be text-only, so you don't need image handling at all. Your wiki might be small enough that the index file is all you need, no search engine required. The right way to use this is to share it with your LLM agent and work together to instantiate a version that fits your needs. The document's only job is to communicate the pattern. Your LLM can figure out the rest.

本文档有意保持抽象。它描述的是一种思路，而非某个具体的实现方案。确切的目录结构、Schema 约定、页面格式、工具选择——所有这些都取决于你的领域、你的偏好，以及你使用的 LLM。上面提到的所有内容都是可选且模块化的——取其有用，舍其无用。例如：你的资料可能全是纯文本，完全不需要图片处理；你的 Wiki 可能足够小，索引文件就已够用，不需要搜索引擎。最好的使用方式是：把这份文档交给 LLM Agent，一起协作，落地出一个适合你需求的具体版本。这份文档唯一的使命，就是传达这个模式本身。剩下的，LLM 会帮你搞定。

小红书
小红书号：95578602331